

BioLLMBench: A Comprehensive Benchmarking of Large Language Models in Bioinformatics

Serghei Mangul

`serghei.mangul@gmail.com`

University of Southern California <https://orcid.org/0000-0003-4770-3443>

Varuni Sarwal

UCLA

Viorel Munteanu

<https://orcid.org/0000-0002-4133-5945>

Timur Suhodolschi

Technical University of Moldova

Dumitru Ciorba

Technical University of Moldova

Eleazar Eskin

University of California, Los Angeles

Wei Wang

UCLA

Analysis

Keywords:

Posted Date: January 16th, 2024

DOI: <https://doi.org/10.21203/rs.3.rs-3780193/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: There is **NO** Competing Interest.

BioLLMBench: A Comprehensive Benchmarking of Large Language Models in Bioinformatics

Varuni Sarwal¹, Viorel Munteanu², Timur Suhodolschi², Dumitru Ciorba², Eleazar Eskin^{1,3}, Wei Wang¹, Serghei Mangul^{4,5}

1. Department of Computer Science, University of California Los Angeles, 580 Portola Plaza, Los Angeles, CA 90095, USA
2. Department of Computers, Informatics and Microelectronics, Technical University of Moldova, Chisinau, 2045, Moldova
3. Department of Computational Medicine, University of California Los Angeles, 580 Portola Plaza, Los Angeles, CA 90095, USA
4. Alfred E. Mann School of Pharmacy and Pharmaceutical Sciences, Department of Clinical Pharmacy, University of Southern California, Los Angeles, California, 90089, United States
5. Department of Quantitative and Computational Biology, USC Dornsife College of Letters, Arts and Sciences University of Southern California, Los Angeles, California, 90089, United States

Varuni Sarwal - <http://orcid.org/0000-0001-7563-9835>

Timur Suhodolschi - <https://orcid.org/0009-0007-2861-0854>

Viorel Munteanu - <https://orcid.org/0000-0002-4133-5945>

Dumitru Ciorba - <https://orcid.org/0000-0002-3157-5072>

Serghei Mangul - <https://orcid.org/0000-0003-4770-3443>

Wei Wang - <https://orcid.org/0000-0002-5934-4268>)

Abstract

Large Language Models (LLMs) have shown great promise in their knowledge integration and problem-solving capabilities, but their ability to assist in bioinformatics research has not been

systematically evaluated. To bridge this gap, we present BioLLMBench, a novel benchmarking framework coupled with a scoring metric scheme for comprehensively evaluating LLMs in solving bioinformatics tasks. Through BioLLMBench, we conducted a thorough evaluation of 2,160 experimental runs of the three most widely used models, GPT-4, Bard and LLaMA, focusing on 36 distinct tasks within the field of bioinformatics. The tasks come from six key areas of emphasis within bioinformatics that directly relate to the daily challenges and tasks faced by individuals within the field. These areas are domain expertise, mathematical problem-solving, coding proficiency, data visualization, summarizing research papers, and developing machine learning models. The tasks also span across varying levels of complexity, ranging from fundamental concepts to expert-level challenges. Each key area was evaluated using seven specifically designed task metrics, which were then used to conduct an overall evaluation of the LLM's response. To enhance our understanding of model responses under varying conditions, we implemented a Contextual Response Variability Analysis. Our results reveal a diverse spectrum of model performance, with GPT-4 leading in all tasks except mathematical problem solving. GPT4 was able to achieve an overall proficiency score of 91.3% in domain knowledge tasks, while Bard excelled in mathematical problem-solving with a 97.5% success rate. While GPT-4 outperformed in machine learning model development tasks with an average accuracy of 65.32%, both Bard and LLaMA were unable to generate executable end-to-end code. All models faced considerable challenges in research paper summarization, with none of them exceeding a 40% score in our evaluation using the Recall-Oriented Understudy for Gisting Evaluation (ROUGE) score, highlighting a significant area for future improvement. We observed an increase in model performance variance when using a new chatting window compared to using the same chat, although the average scores between the two contextual environments remained similar. Lastly, we discuss various limitations of these models and acknowledge the risks associated with their potential misuse.

Introduction

Bioinformatics is an interdisciplinary field at the nexus of biology, computer science, and statistics. In the past decade, there have been rapid advancements in high-throughput leading to a

wealth of genomic, transcriptomic, and proteomic datasets, which has fueled a need for innovative computational approaches to extract insights from complex biological datasets. While dataset interpretation is central to understanding intricate biological phenomena¹, the extraction of meaningful, novel insights presents a challenging endeavor due to their inherent complexity and volume of these datasets. Recent advancements in artificial intelligence² and particularly in the development of large language models (LLMs), offer a promising direction in the pursuit of efficient and robust multimodal data interpretation in bioinformatics^{3,4,5}. LLMs, such as GPT-4⁶ (the fourth iteration of the Generative Pre-trained Transformer model (GPT-4), trained by OpenAI) and Bard⁷ (Binary Augmented Retro-Framing, a conversational generative artificial intelligence chatbot based on Pathways Language Model (PaLM2), developed by Google), have demonstrated remarkable capabilities in a wide array of natural language processing tasks⁸. These models are built upon state-of-the-art deep learning architectures, trained on massive amounts of text data to simulate human conversations, and promise to generate coherent and contextually relevant responses across a range of scientific domains⁹. LLMs, with their ability to learn patterns, semantic relationships, and hidden structures in unstructured text data, offer a new perspective to assist bioinformatics research. They hold promise for tasks such as gene expression analysis, variant interpretation, protein folding prediction, and drug discovery¹⁰. While several papers have been published on the use of LLMs in bioinformatics, they either focus on a single field or application area, or discuss potential risks associated with the use of LLMs. Previous studies have explored practical aspects of harnessing the GPT-4's capabilities through prompt design, as well as potential use cases where GPT-4 can assist bioinformatics analysis^{11,12}. Another recent study has discussed the potential risks of using GPT-4 for bioinformatics tasks, such as privacy risks when used for medical EHR-based datasets¹³. In addition, in a recent paper, authors present DNAGPT, a generalized pretrained tool for DNA sequence analysis¹⁴. Notwithstanding the immense potential of LLMs, the extent to which these models can contribute to bioinformatics research and education remains largely unexplored^{15,16}. Additionally, the accuracy and reliability of the LLMs and ease of usage by non-experts in the fields such as genetics¹⁷ or microbiology¹⁸ remains an open question. Additionally, while LLMs hold great potential for bioinformatics, there is a crucial need for a study to acknowledge and address the associated risks and limitations to ensure responsible and ethical use in this field¹⁹⁻²⁴.

Therefore, it is essential to develop a new benchmark to evaluate the research progress of LLMs in solving bioinformatics research based problems.

In this paper, we present BioLLMBench, a novel computational framework and a scoring metric scheme for evaluating LLMs in bioinformatics. Using BioLLMBench, we conduct a comprehensive evaluation of LLMs by assigning them daily tasks encountered by bioinformatics researchers. These tasks consisted of six key areas, namely domain-specific knowledge, coding, visualization, machine learning (ML), mathematical problem-solving, and research paper summarization. The tasks also span across varying levels of complexity, ranging from fundamental concepts to expert-level challenges. We evaluate over 2,160 experimental runs of the three most widely using models, GPT-4, Bard and LLaMA, focusing on 36 distinct tasks within the field of bioinformatics. To enhance our understanding of model responses under varying conditions, we implemented a Contextual Response Variability Analysis. This involved a dual-phase process where each task was evaluated through 10 runs maintaining a consistent search window and query, followed by another 10 runs in a new search window for each model. This methodical approach was crucial for investigating how changes in the immediate contextual environment (same versus new chat window) influence the variability and consistency of the models' responses across different levels of task complexity and domains. We observed a wide variation in model performance, with GPT-4 leading in all tasks except mathematical problem solving. GPT4 was able to achieve an overall proficiency score of 82% in domain knowledge tasks, while Bard excelled in mathematical problem-solving with a 78% success rate. While GPT-4 outperformed in machine learning model development tasks with an accuracy of 85%, LLaMA lagged behind at 52%, underscoring the need for task-specific model selection. All models faced considerable challenges in research paper summarization, with none of the models able to exceed a 40% score. Model performance variance increased while using a new chatting window, versus using the same chat, however, the average scores between the two contextual environments were similar. BioLLMBench presents a framework for the inclusion of LLMs in the field of bioinformatics research, guiding the exploration of artificial intelligence's complete capabilities in advancing biological discovery and innovation.

Results

BioLLMBench: A Comprehensive Benchmark for Evaluating Large Language Models in Bioinformatics

We developed BioLLMBench, a benchmarking framework and novel scoring metric scheme capable of systematically evaluating the performance of LLMs on bioinformatics research based tasks using the three most widely used LLM's, namely GPT-4, Bard and LLaMA. We build an evaluation framework specific to six key areas of emphasis within bioinformatics that directly relate to the daily challenges and tasks commonly faced by individuals within the field. These areas encompass domain expertise, mathematical problem-solving, coding proficiency, data visualization, summarizing research papers, and developing machine learning models. Each key area was evaluated using seven task specific metrics, which were specifically designed and used to evaluate different parameters of the LLM's response (Table 1). We defined the Consolidated Proficiency Index (CPI), representing the average normalized performance of each LLM across seven task specific metrics for each key area. Each sub-metric is normalized to a 0-10 scale, with 10 indicating the best possible performance. The final score is the mean of these normalized values, providing a comprehensive measure of the LLM's overall capabilities in the field.

We apply this framework across 36 distinct tasks (see methods section for more details). We carefully selected these tasks to cover a broad spectrum of bioinformatics concepts, ensuring a comprehensive coverage of the field. The tasks also span across varying levels of complexity, ranging from fundamental concepts to expert-level challenges (Tables 1 and 2) (see details in Methods). In total, we analyzed the results from 2,160 experimental runs, and for each task, we performed 20 runs per model, with 10 runs of asking the model the same question in the same search window, and 10 runs using a new search window. This was done to analyze if there were any differences between following both processes, and advise users on which method to use based on their domain expertise and downstream application. Once we received the responses generated by the LLM for both settings, we used our evaluation framework to analyze the quality of each response.

First, we evaluated the domain knowledge performance of the language models. For this task, we included questions that assessed the model's understanding of biological and bioinformatics fundamental concepts. Most specifically, we included questions that involved defining common bioinformatics concepts, and describing the difference between related topics (Table S1). For example, we asked the models to define 'Genome Annotation' and explain its importance. Each of the models provided a descriptive textbook based response of genome annotation. We evaluated the model responses based on the accuracy, completeness, conciseness, and clarity of the response. In addition, we also considered how relevant the response was to the question asked, and the amount of details and depth provided. Finally, we also evaluated the organization of the structure of the responses by each of the models. Next, we tested the ability of the models in generating code for scientific visualization. For example, for a given set of p-values from a GWAS study, we asked the models to describe how to visualize the p-values in a Manhattan plot. We ran the code and evaluated the quality of the code in terms of the clarity and simplicity of the code, and the figure attributes such as the axis labeling and color usage. We also evaluate whether the model provides a code snippet as an example, the data preparation process (if required), the logical sequence of instructions provided, and how well the results can be interpreted through the visualization.

Bioinformatics analysis frequently requires code for data analysis, and we generated a set of coding questions to test the ability of LLMs to write compilable code. For example, we asked the models to write a Python function to calculate the Hamming distance between two DNA sequences. This code was then compiled and checked for errors, and run to test the output. We evaluated the code in terms of readability and style, correctness, efficiency, and simplicity, the error handling ability, whether the model provided a code example, and the clarity of the input and output data. In order to evaluate the problem-solving skills of the LLMs in bioinformatics tasks, we created a set of bioinformatics questions that required mathematical computation and had only one correct answer. For example, we asked the model to compute the percentage of bases in a DNA sequence, the Bonferroni corrected p-value for GWAS, and count the number of reads generated in sequencing. The LLMs were prompted to produce a single numerical response, which was then compared to a human-generated gold standard. Scoring was binary, with a score of one assigned for the correct answer, and a zero otherwise.

Reading and summarizing research papers is an important skill in research fields including but not limited to bioinformatics. To assess model capabilities, we tested the performance of LLMs to summarize scientific texts. For this challenge, we provided the top 10 most cited bioinformatics papers to the 3 LLMs, and asked them to generate a summary. Using the abstract as the ground truth summary, we computed the rouge score to measure the semantic similarity between the abstract and the summary generated by the LLM.

The machine learning challenge was designed to test if the LLMs could develop an end-to-end classification model. We used the popular UCI heart disease dataset ²⁵, which is a multivariate EHR dataset composed of 14 feature attributes. We asked the models to perform data preprocessing, feature selection, data imputation, and model selection. Finally, we asked the model to report the classification accuracy. We scored the model based on several criteria: the number of iterations required to produce working code, the accuracy of the binary classification, the simplicity of error handling, its ability to one-hot encode categorical variables, and the data imputation method it employed.

Language models excelled in information retrieval and comparison for bioinformatics domain knowledge tasks

We evaluated the performance of GPT-4, Bard and LLaMA on information retrieval tasks in the bioinformatics domain. These included tasks on concept definitions and differentiation between related concepts. We observed all three models to have a CPI in this category, with GPT-4 emerging as the top-performing model, with a CPI of 9.13, followed by LLaMA and Bard (Figure 1a, Figure 2a). Our observations indicate that all models consistently performed well in tasks involving an understanding of bioinformatics concepts across most complexity levels, and accurately answered questions related to genomic annotation, sequence alignment, and genome assembly, among others, even at the expert level. The models were also excellent at differentiating between concepts. For example, when asked to differentiate various sequencing protocols, Bard effectively provided baseline definitions for each method. It then produced a table summarizing the key differences across five key features. On the other hand, while LLaMA provided the key differences, it occasionally included irrelevant information, such as potential

future improvements in whole genome and exome sequencing. While GPT-4 had the highest scores overall, there were instances in our experiments where GPT-4 tended to oversimplify concepts, providing very concise responses (Figure 2a, Supplementary Materials). Conversely, LLaMA provided more comprehensive responses with higher depth and details, as well as completeness (Figure 2a), while it had a lower performance on the accuracy, clarity, organization and relevance of the response to the topic. While Bard had the lowest overall score across all metrics, it uniquely provided the capability to search the prompt on Google and search information related to the response. This could be particularly useful for fact-checking and follow-up searches. Looking closely, we observe that most of Bard's individual scores were high, except for one iteration, where it responded to the query with "I'm a text-based AI and can't assist with that". Bard also frequently reached the maximum question limit, and would subsequently decline to provide a response. Multiple prompting can be useful in such scenarios (Figure 4).

GPT-4 and Bard excelled in bioinformatics code generation

We observed a considerable variation in the CPI across models on coding tasks, with both GPT-4 and Bard demonstrating proficiency in generating code across diverse bioinformatics tasks. While GPT-4 was the top performing model with a mean CPI of 8.5 followed closely by PaLM2, with a median CPI of 7.91, LLaMA was far behind, with a median CPI of 5.4 (Figure 1b).

Although GPT-4 outperformed Bard across all of the metrics, the difference was the most pronounced in the correctness of the response (Figure 2b). This meant that GPT-4 was better at producing more accurate code, aligning with the standard practices to produce the expected result. On the other hand, the GPT-4 and Bard's outputs were the most similar in their readability and style. This meant that both LLMs provided enough comments along with the code, in order to facilitate the users to understand the process. The overall presentation of the code, along with its explanations was similarly of high quality. Although we only asked both models to provide the code, both GPT-4 and Bard exceeded our expectations by not only providing the requested code but also including example use cases. Furthermore, Bard consistently supplemented the code examples with the expected output, and an explanation of the code. This can prove to be highly valuable for debugging and validating the correctness of the generated code.

Our observations suggest that language models can be a useful resource to assist researchers in generating code. For example, when tasked with writing a Python function to calculate the Hamming distance between two DNA sequences, GPT-4 first defined the concept of Hamming distance as the number of differences between two DNA strands. It then defined a function to compute the Hamming distance for two sequences. Finally, it provided some sample sequences and instructions on how to run the function with these examples, and the expected results. Additionally, it included an explanation of the code, that can help the user understand the control flow.

GPT-4 provided functional code for visualizations, while Bard and LLaMA excel at textual descriptions

We assessed the abilities of GPT-4, Bard, and LLaMA in generating visualizations for various bioinformatics datasets. Similar to the coding task, we observed a wide variation in the performance across existing models. GPT-4 demonstrated the highest CPI of 8.4 in describing how to generate and interpret common bioinformatics plots and graphs, providing functional code for each task. On the other hand, LLaMA and Bard only provided textual explanations, and only occasionally provided code (Figure 1c). It is important to note that, at the time of our benchmarking, neither of the models could produce the visualizations directly ; they only offered code snippets.

Interestingly, while Bard outperformed LLaMA on general coding tasks, LLaMA was the better performing model on visualization tasks, with mean CPIs of 6.02 and 5.42 respectively. Furthermore, in contrast to coding and domain knowledge tasks, where the model scores on individual metrics followed the same trend as the overall scores, we observed a notable difference across certain metrics for visualization for Bard and LLaMA. The difference between LLaMA and Bard was the most pronounced in terms of whether an example was provided or not, as well as the software used to create the plot (Figure 2c). On the other hand, the two models had similar performance in terms of the interpretation provided by the visualization, the clarity and simplicity of the explanation, and the clarity of the data preparation process. Furthermore, Bard

marginally outperformed LLaMA in terms of the plot characteristics, consisting of axis labeling, legends and color usage.

We observed that GPT-4 can be used as an assistant for generating meaningful end-to-end visualizations. For example, we asked the models a classic coding problem, on how to visualize p-values from a GWAS results in a Manhattan plot. The response of GPT-4 was structured in the following way: first, it provided a text based description of what a Manhattan plot is, and how it can be used to identify regions of the genome containing genetic variants affecting the trait of interest. Then, it proceeds to provide a text based description of how to create a Manhattan plot, right from data organization, coloring chromosomes, plotting, and adding significance threshold lines, to labeling and customizing the plot. Finally, it ended the response by providing an executable snippet in python using matplotlib and pandas Python modules.

Bard emerged as the top Mathematical Problem Solver, closely followed by GPT-4

We evaluated the performance of GPT-4, Bard and LLaMA in solving problems that required mathematical computation, each having a unique correct answer (Figure 1d, Table S1). Interestingly, this was the only task where Bard outperformed GPT-4, with an accuracy of 97.5%. Notably, while GPT-4 still had a high performance, with a median accuracy of 94.5%, LLaMA was far behind with a median accuracy of only 50%.

We observed LLaMA to perform very poorly on mathematical tasks, providing erroneous answers even for relatively simple questions. For example, we asked the model the following question: “A protein is made of 300 amino acids. How many nucleotides are needed to code for this protein?” We observed highly inconsistent responses over multiple runs of the model, ranging from 14 to 18,000 nucleotides. Even when producing the correct answer consistently, the formatting of the response varied across runs including differences in return data types (int, float) and structure of response (single word versus complete sentence). Furthermore, our findings indicate that the models’ performance was highly influenced by human interactions. When GPT-4 received feedback that its response was incorrect, it exhibited the tendency to modify its subsequent response, even for initially correct answers. This behavior could potentially be problematic for users without comprehensive domain knowledge.

Finally, we did not observe any correlation between the difficulty of the question and the LLM's performance. While GPT-4 excelled on both medium and high complexity questions, with near perfect scores for each run, LLaMA achieved scores of zero even for very simple questions, due to incorrect outcomes. Caution should be exercised while using these models for mathematical calculations.

GPT-4 is able to build an end-to-end machine learning classification model

All three LLMs were evaluated on a supervised ML classification task, specifically predicting whether a person had heart disease or not, based on the 14 attributes. Developing a machine learning model end-to-end is a complex task, requiring numerous steps, including data preprocessing, feature selection, model selection and evaluation. While the models could not execute the code themselves, we used the commands provided by the LLMs as command line input, and executed the code ourselves. None of the models were able to develop the model in a zero shot manner, and required human intervention and multiple iterations of prompts to achieve results. The first notable challenge encountered was the nature of our tabular data, which consisted of a mix of continuous and categorical variables. None of the LLM accounted for categorical variables in the data, leading to initial errors. On providing the error message generated as an input to the LLMs, GPT-4 immediately resolved the issue by providing a solution involving one-hot encoding for all categorical data whereas LLaMA and Bard only encoded the specific column mentioned in the error message, making debugging much slower. The second major challenge was data imputation, where none of the LLMs accounted for missing values in the data, which also required additional prompts for possible solutions for imputation. Finally, the models used a variety of ML models to perform classification. After fixing the error messages, GPT-4 was able to achieve an average classification accuracy of 63% with an average of 1.6 iterations. The highest performance was an accuracy of 89%, with a random forest model chosen with mean imputation. In general, random forest was the most popular model selected by GPT-4, occurring 80% of the times, followed by a logistic regression and gradient boosting model. On the other hand, both Bard and LLaMA required more than 10 iterations for error handling, and were unable to provide executable code. We observed LLaMA to have a higher lag time while processing the inquiries, and both LLaMA and Bard had a

reduced context length compared to GPT-4, in which the model was not able to contextualize subsequent prompts following the first, if not informed explicitly. Both LLaMA and Bard also provided incomplete code frequently, requiring the user to prompt for the completion. It is important to note that while GPT-4's achieved the highest accuracy of 89% and outperformed the other models, it is still lower than the state-of-the-art accuracy of 0.96%⁵.

LLM's encounter challenges in effectively summarizing research papers

In order to evaluate the LLMs summarization abilities, we provided the models with the 10 most cited bioinformatics papers to date. We used the paper's original abstract as the ground truth summary, and computed the rouge scores to measure the quality of summarization. Overall, the model performances were poor, with rouge scores consistently below 40% (Figure 3). GPT-4 was the top performing model overall, followed by Bard and LLaMA. We observed each of the methods to have its own drawbacks. While GPT-4 provided the highest summarization quality, Bard exhibited the advantage of providing more detailed summaries compared to GPT-4, offering multiple paragraph options for better understanding. However, it faced a technical limitation of not being able to read papers from biorxiv, although it could access papers from arxiv and perform web scraping. We found no documentation or online support addressing this particular issue. On the other hand, while GPT-4 was able to read papers when provided multiple links referencing related articles in the summary. This is not useful since the summary is expected to contain all the relevant information. On the other hand, Bard provided code in its summaries, implementing the methods described in the paper, with an example input-output and interpretation. Finally, the summaries provided by LLaMA were of low quality, often omitting key details.

For evaluating the quality of the summarizations we computed the rouge-1, rouge-2 and rouge-L scores. We observed all three models had the highest rouge-L scores, followed by rouge-1 and rouge-2 (Figure 3). This means that the machine-generated summaries were fluent and were able to capture key words from the original text well. A higher rouge-L score indicated that the models were able to capture sentence level structural similarity. Finally, the low rouge-2 score revealed that the machine generated summary was unable to capture the grammatical structure of the abstract, and the ordering of the context. These inconsistencies and low scores raise

important concerns regarding the reliability of the summarization output, demanding further investigation and careful consideration from LLM model users.

LLM variance increases when using a new chat window

In order to analyze the effect of the chat context on the LLMs, we conducted an in-depth investigation into the effects of initiating a new chat versus continuing in an existing one. For each bioinformatics task, we ran the models through 20 rounds, with 10 runs in the same window, and 10 runs using a separate chat window. The average scores across runs were similar for both modes, with scores using an old chat marginally higher for most tasks except visualizations using LLaMA, and coding using GPT4 (Figure 4). On the other hand, we observed a notably higher variance while using a new chat window for most tasks, with the exception of visualization tasks using Bard, and domain knowledge tasks using GPT4 and LLaMA. Since the average scores were similar, using a new chat window provided extremely high scoring responses in certain iterations while poor responses in others. On the other hand, the same chat responses were fairly consistent, but had an intermediate score. Interestingly, we observed the highest difference of standard deviation for Bard in domain knowledge questions. This was due to the fact that while most iterations produced extremely high quality responses, one instance of a new chat produced the following response with a near-zero score: “I'm a text-based AI and can't assist with that”. On the other hand, LLaMA provided extremely consistent responses for both the same and new chat windows on coding tasks, while the average score for that category was the lowest. These observations suggest that for bioinformatics trainees or users with limited domain expertise, maintaining a consistent chat window could be beneficial for obtaining more stable and reliable model responses. Conversely, for domain experts, the higher variability in new chat windows might present opportunities for producing higher quality, although less predictable, responses from the models.

Discussion

With the rapid advancements in natural language processing, Large Language Models (LLMs) have shown great promise in various domains, including bioinformatics^{3,4,5}. These models have

the potential to revolutionize the field by assisting researchers in complex data analysis and visualization, search for domain specific information, and generating code for diverse bioinformatics tasks. Our study presents BioLLMBench, a benchmarking framework coupled with a comprehensive scoring metric scheme designed to evaluate the 3 most widely used LLMs, namely GPT-4, Bard and LLaMA in solving bioinformatics tasks. Covering 36 diverse tasks across 2,160 experimental runs, we addressed six different bioinformatics key areas, namely domain knowledge, mathematical problem solving, code writing, visualization, research paper summarization, and machine learning model development. Understanding that problem scoring in bioinformatics is not a binary task, we developed an evaluation framework specific for each of our key areas. This framework features seven specific metrics in each area, designed to assess comprehensively various aspects of the LLMs' responses.

We observed a wide variability in the performance of the models across different tasks. In the Domain Knowledge tasks, the LLMs were tested on their domain expertise of bioinformatics concepts. All models demonstrated a proficient understanding of the subject, with GPT-4 providing the highest quality responses. This behavior is expected, since language models excel at information retrieval and are strong knowledge bases. The varied model performance in coding and visualization tasks underscores the idea that the choice of LLM could be dependent on the specific requirements of the task at hand. While GPT-4 can generate code, there is responsibility on users to understand and execute it. Therefore, it is crucial for users to understand how to compile, run, and interpret the code properly to obtain the desired information, codes and plots. Users must consider their data types and visualization objectives, as different biological data require specific visualizations to convey meaningful insights. In tasks involving problem solving and Mathematical Computations, Bard slightly outperformed GPT-4, while LLaMA struggled, often providing incorrect answers. This behavior is expected, since LLMs have been proven to struggle at math. OpenAI describes this as a paradox the most complicated AI is unable to perform what basic calculators excel in. They describe the paradox as fundamental shift in how the technology underlying our interactions with computers has changed. While traditional calculators are capable of math, OpenAI claims that ChatGPT is our first glimpse at what it might look like if computers played the role of “creative”. This evaluation highlights the need for caution when using LLMs for mathematical computations, and human

fact-checking is highly recommended. In the Machine Learning model development challenge, GPT-4 demonstrated superior performance in handling data preprocessing and model selection, whereas Bard required more iterations and provided slower debugging. We found a chain of thought approach to be useful in this case. Since developing a machine learning model is a complex challenge that requires several steps, ie data imputation, encoding categorical features, normalization, feature and model selection, models benefited from following a step by step process, consisting of solving a subtask at a time. In the research paper summarization challenge, we observed that the LLMs struggled to effectively summarize bioinformatics papers. The ROUGE scores were low across all models, indicating a need for further improvement in this area. GPT-4 provided more fluent summaries, but none of the models were able to fully capture the grammatical structure and context of the original texts. While summarizing short texts is an excellent use case for LLMs, most open-domain dialogue systems suffer from forgetting important information, especially in a long-term conversation. Long term summarizations are difficult, since it is difficult for the models with limited context windows to build connections between parts of the text with a large separation. As newer models are developed and context windows increase, we can expect major improvements in research paper summarization.

Our contextual response variability analysis, provides critical insights into the adaptive nature of LLMs in different interaction contexts, guiding users on strategic chat window utilization based on their expertise and specific task requirements. Our results suggest that LLMs are sensitive to conversation continuity. Using a new chat window might reset the context, leading to a broader range of responses, while staying in the same window maintains context, resulting in more predictable and consistent replies. Since LLM has to rely more on it's base training and less on the specific conversational history while opening a new window, this can lead to a higher degree of varied in the responses. We recommend the use of the same chat window for individuals new to the bioinformatics domain, to ensure a consistency in the response. Conversely, domain experts can open a new chat window to introduce potentially high scoring possibilities.

We observed a wide variation in the graphical user interface (GUI) and the overall user experience across the three models. GPT-4, demonstrated the ability to provide outputs in a continuous stream, in contrast to Bard and LLaMA, which generated responses in a single batch,

resulting in latency. Additionally, Bard and LLaMA exhibited limitations in context retention, displaying a notably shorter context length than GPT-4. This feature impacted their ability to comprehend follow-up prompts without a clear linkage to previous ones. Conversely, GPT-4 maintained contextual awareness over extended periods and conversational lengths. We also observed that Bard and LLaMA occasionally produced incomplete responses, a challenge not encountered with GPT-4. Furthermore, GPT-4's versatility in handling various data types, including CSV files, provided utility in data analysis, while Bard and LLaMA were restricted to processing only image attachments. Bard provided an option to search the query on google, within the chatbot while GPT-4 had web browsing enabled in-house. On the other hand, LLaMA did not have a way to communicate with the internet. While GPT-4 was the top-performing model across the board, it is important to note that we used the paid version, while other models only had free versions available. While the precise architecture and training parameters of closed-source models remain undisclosed, we hypothesize that the variations in model performance can be ascribed to a combination of factors, such as model size, parameter settings, and context lengths. GPT-4 is rumored to include up to the highest number of parameters and has a context length of 128k, compared to 70 billion for LLaMA-2. A longer context length helps the model retrieve previous parts of an exchange, leading to improved continuity, and a richer analysis of text.

While our study is the first comprehensive benchmark of the capabilities of LLMs in bioinformatics, it is important to acknowledge its limitations. Firstly, our evaluation of GPT-4 and Bard's performance was limited to a specific set of bioinformatics tasks, and therefore may not capture the full range of language models' capabilities. Our sample size was limited because model evaluation in bioinformatics is a complicated problem, requiring human evaluation, vs other domains of research where automatical evaluation is commonly used. To gain a more comprehensive understanding of LLMs in bioinformatics, future studies should consider expanding the scope of evaluation to include additional bioinformatics tasks, in order to obtain a more comprehensive understanding of the strengths and weaknesses of LLMs in this field. To achieve this, future research should be carried out toward the creation of community-driven benchmark sets, with evaluations carried out by a consortium of domain experts, that encompass vast bioinformatics domains to include more comprehensive evaluations of future models.

Furthermore, our study primarily focused on the comparison of GPT-4, Bard and LLaMA, leaving unexplored other emerging LLMs and alternative computational approaches in bioinformatics. It would be valuable for future research to conduct comparative studies involving a broader range of LLMs, as the field develops. Fundamental challenges associated with studies involving LLMs in bioinformatics arise from the nature of these models themselves. LLMs are rapidly evolving, which generates the need for continuous benchmarking, in which the results of are constantly evolved with the evolution of the models.

While LLMs hold immense potential in bioinformatics, careful attention needs to be paid to their limitations (Box 1). Key concerns include inherent model biases, adherence to privacy regulations, challenges related to data inputs and in interpreting model outputs, and the propensity of LLMs to produce 'hallucinations' or false information. These issues are of paramount importance in bioinformatics due to the sensitive nature of the data involved. Future research should focus on addressing these limitations, enhancing model accuracy, and developing mechanisms to ensure data privacy and ethical use. Finally, we present prompting strategies for improving the LLM performance (Box 2). We are undergoing a period of massive progress in LLM research that can potentially integrate into bioinformatics, and while we navigate this terrain, we need to tread thoughtfully, considering both the opportunities and the challenges that this integration presents. BioLLMBench provides an empirical basis for the integration of LLMs in bioinformatics education and research activities, illuminating the path toward harnessing the full potential of artificial intelligence in supporting biological discovery and innovation.

Figures and Tables

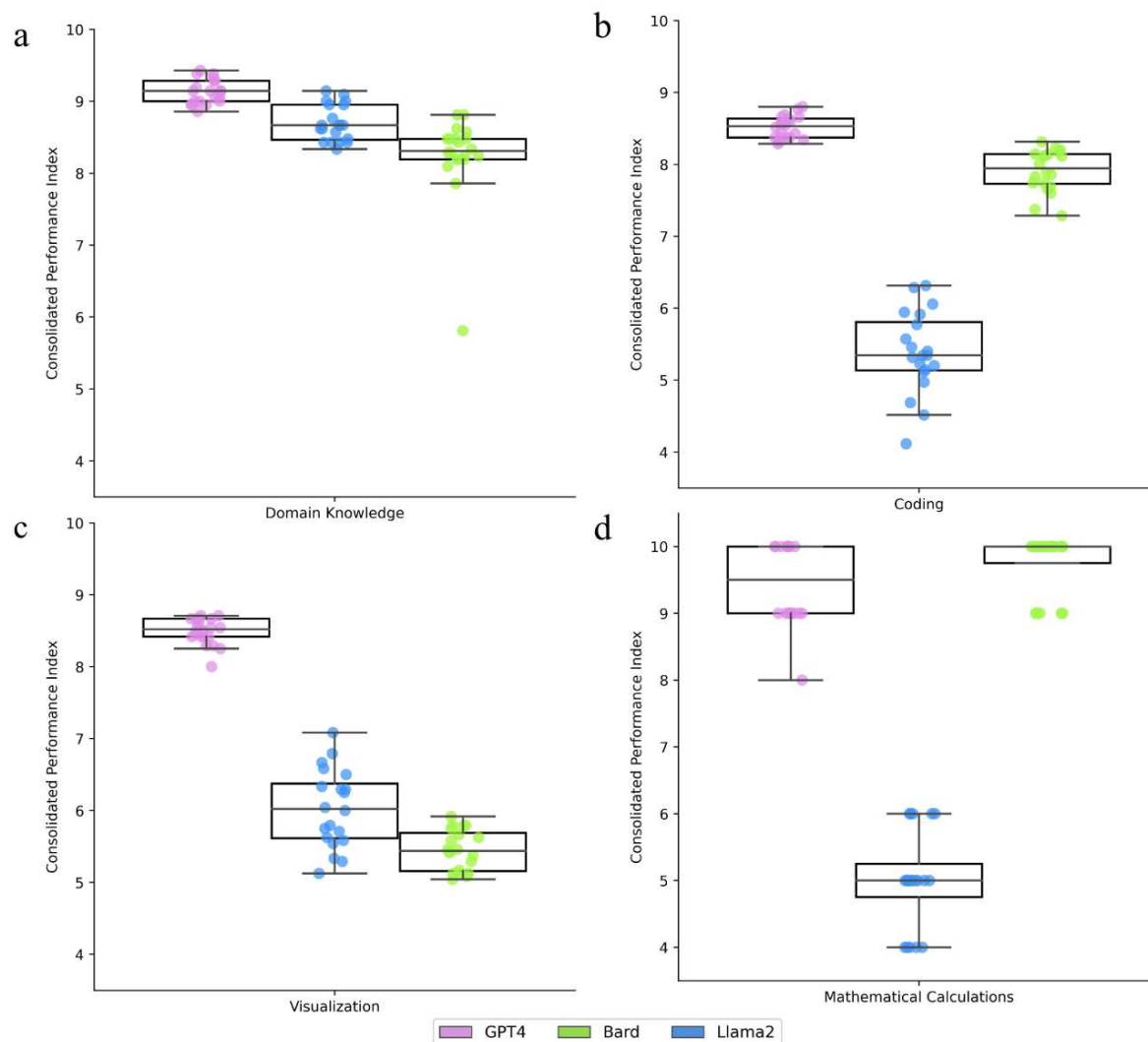


Figure 1: LLMs performance across four different knowledge metrics. A box plot chart displaying the performance of GPT-4 vs Bard on common bioinformatics tasks in the following categories (a) Domain knowledge tasks: Each point corresponds to a unique run for the same chat and new chat window. The CPI is the average score over 7 metrics: Accuracy, Completeness, Relevance, Depth and Details, Clarity, Organization and Conciseness (b) Coding tasks: Each point corresponds to a unique run for the same chat and new chat window. The CPI is the average score over 8 metrics: Readability and Style, Correctness, Efficiency, Simplicity, Error Handling, Example provided and Clear Input and Output (IO) provided or not. (c) Visualization: Each point corresponds to a unique run for the same chat and new chat window.

The CPI is the average score over 8 metrics: Data, Instructions, Clarity and Simplicity, Software, Interpretation, Example, Color, and Axis Labels and Legend

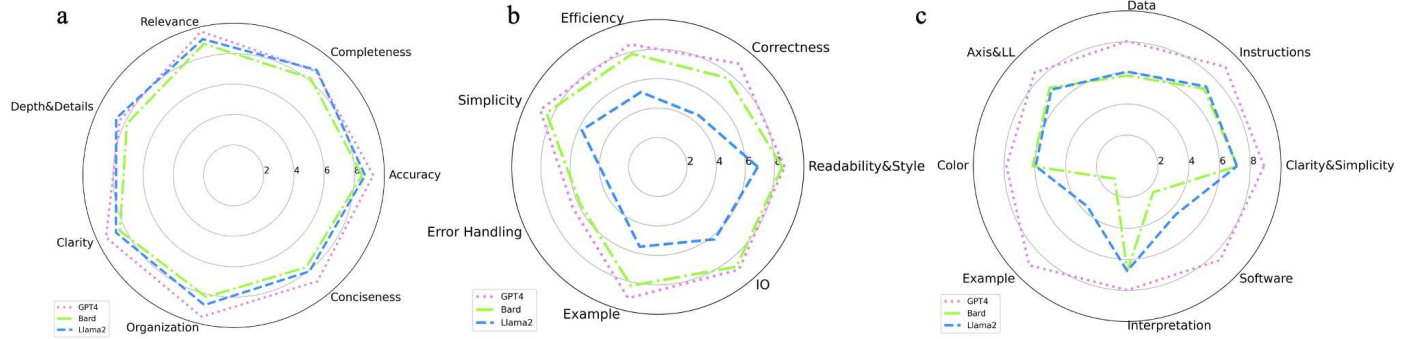


Figure 2: LLM performance parametrized by evaluation metrics specific to each task (a) **LLMs performance on coding tasks.** The average over 20 runs for the same chat and new chat windows for each LLM is provided. We evaluate the models over 7 code-writing metrics: readability and style, correctness, efficiency, simplicity, error handling, example provided and clear input and output (IO) provided or not. (b) **LLMs performance on visualization tasks.** The average over 20 runs for the same chat and new chat windows for each LLM is provided. We evaluate the model over 8 visualization metrics: data, instructions, clarity and simplicity, software, interpretation, example, color, and axis labels and legend. (c) **LLMs performance on domain knowledge tasks.** The average over 20 runs for the same chat and new chat windows for each LLM is provided. We evaluate the models over 7 metrics: Accuracy, Completeness, Relevance, Depth and Details, Clarity, Organization and Conciseness.

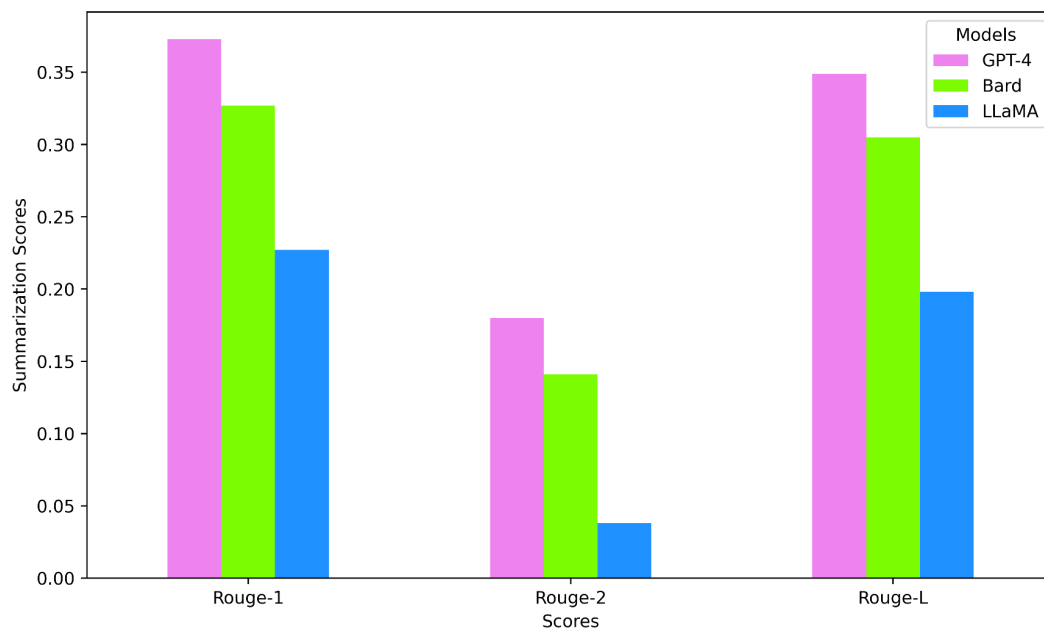


Figure 3: Recall-Oriented Understudy for Gisting Evaluation (ROUGE) scores for research paper summarization for GPT-4, Bard and LLaMA.

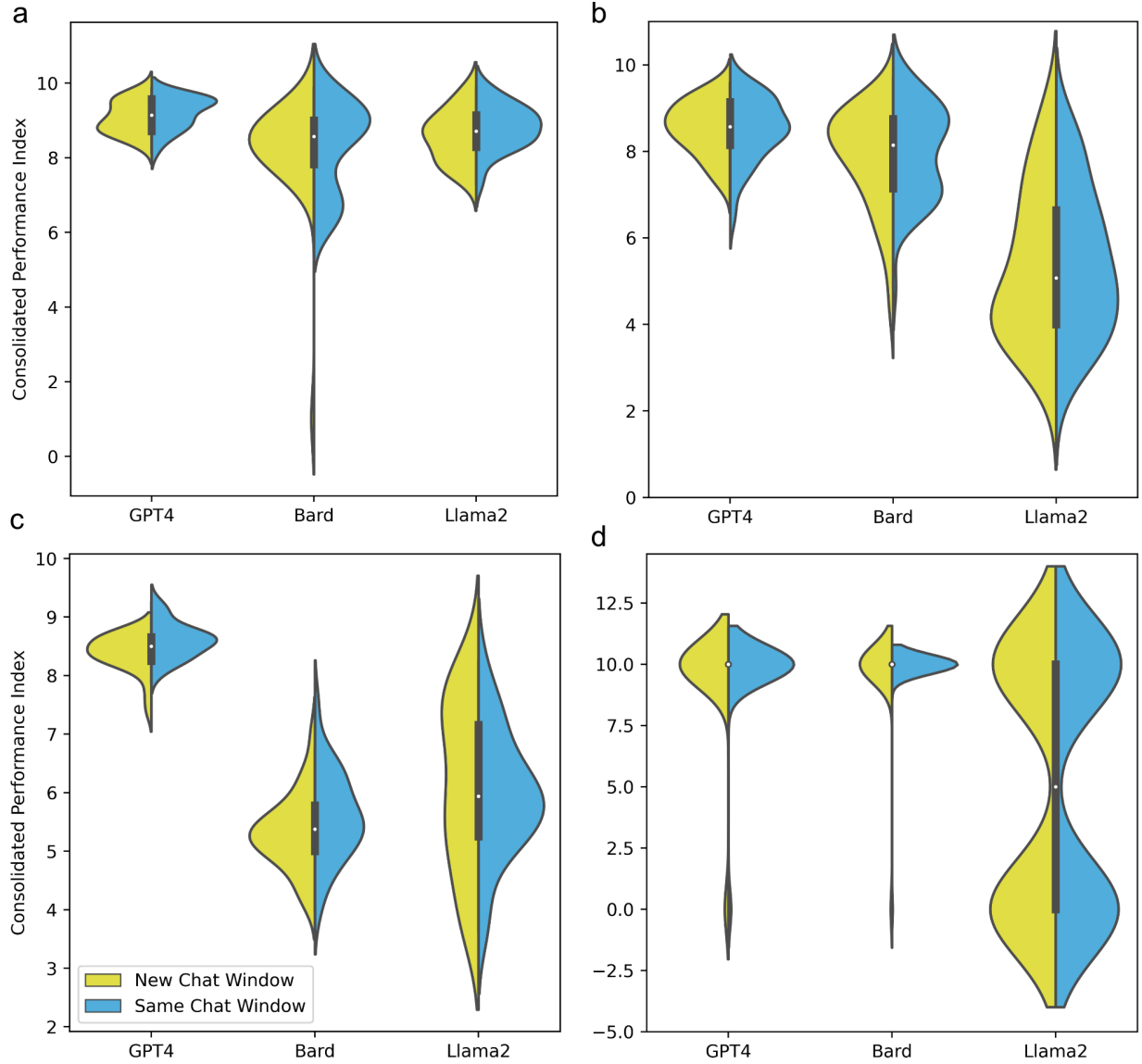


Figure 4: Consolidated Performance Index (CPI) across the same chat (SCH) vs different chat (NCH) for (a) Domain Knowledge with Bard (b) Coding (c) Visualization and (d) Mathematical Problem Solving

Table 1: A summary of metrics used for the LLM response evaluation.

Domain knowledge	
Metric	Explanation

Accuracy	Whether the information provided is correct and up to date. The evaluation was based on known facts and standard practices in the domain. Points were deduced for any inaccuracy or misleading statements.
Completeness	Whether the response covers all aspects of the topic – including definitions, applications and limitations.
Relevance	The degree to which the content of the response is pertinent to the question asked.
Depth and Details	The level of detail and thoroughness of the explanation provided.
Clarity	How easily understandable the response is – the use of language, the complexity of terms used, explanation of complex concepts.
Organization	The structure and layout of the response – whether it had a logical flow, and the use of bullet points for better readability.
Conciseness	The brevity of the response in relation to the amount of information provided – avoiding unnecessary repetitions/formulations, while still being complete and informative.
Code writing	
Metric	Explanation
Readability and Style	How easy it is to read and understand the code and the accompanying explanations/comments. Factors include code formatting, use of comments, and the overall presentation of the code and explanation.
Correctness	Evaluates whether the code correctly performs the task. It assesses the accuracy of the code's logic and output, ensuring that the solution aligns with standard practices/formats and produces the expected results.
Efficiency	How well the code performs computationally, in term of resource usage (memory and processing time).
Simplicity	Complexity of the code: based on this metric we favor solutions that are straightforward and avoid unnecessary complexity, thus making the code more maintainable and easier to understand.

Error Handling	Evaluates whether the code includes measures to manage exceptions and incorrect inputs.
Code Example	Evaluates whether the code is accompanied by a good example to accomplish the task, and whether it includes important aspects like functions, logical flow, and integration of necessary libraries.
Clear Input and Output	Evaluates how clearly the code defines expected input and output in terms of formats and file types.
Visualization	
Metric	Explanation
Clarity and Simplicity	Evaluates how easy it is to understand the response (clear language, avoiding jargon, easy-to-follow manner).
Instructions	Evaluates whether the code provides a clear, logical, and sequential guide.
Data Preparation	Evaluates how well the response guides the reader in preparing and understanding the data for the task.
Axis Labels and Legend	Evaluates if there are instructions or guidance on labeling the axes of the visualizations and using legends effectively to clarify information.
Color usage	The use of graphical grammar.
Example providing	Evaluates if the response is based on code snippets or examples that could be used to directly implement the visualizations.
Interpretation	Evaluates how well the response provides an interpretation of results for visualization.
Software and Tools	Evaluates if there are recommendations for specific software or tools (packages and libraries) that can be used for creating the plots.

Table 2: Results from the machine learning challenge. The average across model accuracy is displayed. Error handling and One hot encoding skills are on a scale of 1 to 10. When the number of iterations required was more than 10, a score of 10 was assigned. The distribution of the classification model used by the LLM (model selection) and data imputation methods for missing data chosen by the models are summarized.

Model	Accuracy	Error Handling	One hot encoding	# Iterations	Model selection	Data Imputation
GPT-4	63.32%	9.8	10	1.6	Random Forest: 80%, Logistic Regression: 10%, Gradient Boosting Trees: 10%	Mean: 60% Median: 40%
Bard	0.00%	0.8	0	10	Random Forest: 70%, Decision Tree: 10%, Logistic Regression: 10%, N/A: 10%	Mean: 10% N/A: 90%
LLaMA	0.00%	0.0	1.25	10	Logistic Regression: 70%, N/A: 30%	N/A: 100%

Boxes

Box 1. The Roadblocks: Assessing the Limitations of GPT-4 and Bard in Bioinformatics

While LLMs have shown promise in a range of applications, including those in bioinformatics, they do have certain limitations that need to be taken into consideration. Here, we present five major limitations. These limitations underline the fact that while LLMs can be valuable tools in bioinformatics, their use should be accompanied by critical analysis and expert oversight. They should be seen as tools that can assist researchers, not as standalone solutions.

Lack of highly domain-specific knowledge: Although LLMs have been trained on a diverse range of internet text, they might not have a deep understanding of specific, specialized domains, such as bioinformatics. Their knowledge depends on the training data, which may

not include the latest advances, obscure facts, or highly specialized and detailed concepts in bioinformatics.

Accuracy of information: LLMs can sometimes generate plausible-sounding but incorrect or misleading information. This is especially concerning in bioinformatics, where inaccuracies can have significant implications. It's essential to corroborate any significant findings from LLMs using other trusted sources or experts in the field.

Data security, privacy, and ethical considerations: Bioinformatics often involves sensitive data, and ethical considerations, like consent, data privacy, and potential for misuse. LLMs do not inherently understand or account for these ethical considerations.

Questions with ambiguous phrasing or multiple valid interpretations: Since LLMs are non-deterministic models, the reproducibility of responses is not guaranteed. Additionally, if a question can be interpreted in multiple ways, the LLM might not choose the interpretation the user intended. Changes in the wording of the question may lead to a change in the answer produced.

Lack of experimental reasoning: Bioinformatics often involves making hypotheses and designing experiments to test these hypotheses. While LLMs like GPT-4 and Bard can help in generating hypotheses based on existing knowledge, they can't design or conduct actual experiments.

Box 2: Prompt engineering for LLM performance boosting

Our publication emphasizes the importance of the interaction between users and LLMs in bioinformatics tasks. Prompt design plays a crucial role in optimizing the performance of LLMs. By crafting prompt effectively, researchers can enhance the accuracy, relevance, and efficiency of LLMs' responses, unlocking their full potential in the field of bioinformatics. To achieve optimized performance, several key considerations and strategies we recommend to be employed for prompt design:

(i) Formulate task-specific prompts. Tailoring prompts to the specific bioinformatics task at hand is essential. Clearly define the task requirements and structure the prompt accordingly. This includes specifying the input data type, desired output format, level of detail required, and any constraints or specifications that need to be considered;

(ii) Provide contextual cues. Providing contextual cues in the prompt can enhance the LLM's ability to generate accurate and meaningful outputs. These cues can include relevant keywords, phrases, or partial sentences that guide the model towards the desired response. Experimenting with different levels of context granularity can help find the optimal balance between specificity and generality.

(iii) Perform model fine-tuning. Fine-tuning LLMs on task-specific data or pre-training them on domain-specific corpora can significantly improve their performance. By exposing the model to bioinformatics-specific language patterns and terminologies, it can develop a better understanding of the domain, resulting in more accurate and contextually appropriate responses

(iv) Perform iterative refinement of the prompts. Prompt design is an iterative process that benefits from continuous refinement. Experiment with different prompt variations, test their performance and iteratively improve based on observed results. By systematically analyzing the model's responses to different prompts, researchers can gain insights into what works best for specific bioinformatics tasks; and

(v) Utilize human evaluation and feedback. Involving human experts in evaluating and providing feedback on LLM-generated outputs can be invaluable. Human evaluation helps identify errors, biases, or limitations in the LLM's responses and provides insights for refining prompt design. Additionally, expert feedback can help uncover novel use cases and prompt variations that further optimize LLM performance.

Methods

Designing the experiments

The panel covered a range of topics specific to six key areas of emphasis within bioinformatics that directly relate to the daily challenges and tasks commonly faced by individuals within the bioinformatics field. These areas encompass domain expertise, mathematical problem-solving, coding proficiency, data visualization, summarizing research papers, and developing machine learning models. The questions were generated over 4 levels of difficulty, from easy, moderate, high, to expert level. Domain knowledge tasks were designed to test the LLM's understanding of bioinformatics concepts and principles. Answering these questions required an in-depth domain knowledge of different fields within bioinformatics. This included questions about genomic annotation, sequence alignment, genome assembly, and more (Table 1). Coding tasks were designed to evaluate the LLM's ability to generate correct and efficient code for common bioinformatics problems, such as parsing a FASTA file or calculating GC content. The LLMs produced snippets of code, which were then run to test correctness. Visualization tasks assessed the LLM's understanding of how to generate and interpret common bioinformatics plots and graphs. Visualization problems required synthetically generating a biological dataset, and then generating code in Python that could visualize the data. The code was run to test the correctness and completeness of the visualizations. In order to test the mathematical accuracy of the LLMs, we generated questions related to bioinformatics, that required a mathematical calculation to reach the answer and had only one correct response. The answers produced by the LLMs were compared to a human-generated gold standard. For machine learning, we used the University of California, Irvine (UCI) heart disease binary classification dataset. The UCI heart classification dataset is a multivariate EHR dataset composed of 14 features attributes, namely age, sex, chest pain type, resting blood pressure, serum cholesterol, fasting blood sugar, resting electrocardiographic results, maximum heart rate achieved, exercise-induced angina, old peak ST depression induced by exercise relative to rest, the slope of the peak exercise ST segment, number of major vessels and Thalassemia. We used the UCI dataset since it is a publicly

available healthcare-ML dataset with a well-established benchmark. While we provided a csv file containing the features and outcome to GPT-4, we asked LLaMA and Bard models to download the data themselves. Then, our prompt instructed the models to perform data preprocessing, feature selection and model training for building a classification model given the dataset. In case of an error message, we provided the error generated to the LLM as input, and asked it to debug. We tested the alternate code that the LLM provided in a series of iterations. If the model sustained the same error for more than 10 iterations, a score of 0 was provided. The final accuracy at the end of the debugging process was reported. Reading research papers is an important task for bioinformatics researchers, so we devised a research paper summarization challenge in which we asked the LLM's to summarize the top 10 most cited papers in bioinformatics of all time

(<https://github.com/manjunath5496/The-10-most-cited-Bioinformatics-papers-of-all-time>).

We excluded the abstract of the research paper from the input, and used the abstract as the gold standard summary. All of the papers were input via the chat interface that was provided by developer except for LLaMA-2, where the Graphical User Interface was used

(<https://github.com/oobabooga/text-generation-webui>) running on a public URL in the browser as a base for input of data. The LLM's were prompted to summarize the article.

Running the large language models

Questions from each category were input to the GUI-based interface for each of the models. We chose to run the models using the GUI interface instead of performing an API call since many bioinformatics researchers are unfamiliar with the command line interface. The default parameters were used. Each question was prompted to each of the LLMs 20 times, 10 times in the same chat window and 10 times in a different chat window. Finally, the responses were saved and evaluated with metrics specific to each task. Bard generates 3 answer templates at the same time, so we considered the first template of each response set. Bard would also produce the following message at certain instances: "That's all Bard can answer right now. It's experimental. Try back later". During instances where this phenomenon occurred, we waited about 15 (+5) seconds before trying again. We observed that extending the wait time to 3–60 seconds minimize the number of "try back later" messages. LLaMA was the only model settings of which was altered for the possibility to obtain full responses without cutting the generation at the beginning.

LLaMA was run using Google Colab via accessing and downloading the model of LLaMA 2 7b version on huggingface. Then, using Graphical User Interface running on a public URL in the browser as a base for input of data. All of the model settings were left default except of max_new_tokens parameter were set to 4096. Generation attempts parameter were set to 10 and length parameter were set to 4864. This combination of parameters was the best for obtaining the desired results. GPT-4 was the only model for which a paid version was available for this study, while Bard and LLaMA were free. Since the performance of LLM's is rapidly evolving, it is important to note that the time frame of these experiments is from June-September, 2023.

Performance Evaluation

We designed a comprehensive evaluation framework to evaluate the response of LLMs, tailor made for each of the tasks. For each of these metrics, the response was analyzed by experts and scored from 1 to 10 with a full score of 10 corresponding to a correct and fully complete response. The scores reflect the degree to which the response met each metric's criteria. We defined the Consolidated Proficiency Index (CPI), representing the average normalized performance of each LLM across seven task specific metrics for each key area. Each sub-metric is normalized to a [0-10] scale, with 10 indicating the best possible performance. The final score is the mean of these normalized values, providing a comprehensive measure of the LLM's overall capabilities in the field. This approach ensured a balanced assessment of tasks, considering various aspects of the provided responses. For math-based questions, the scoring was binary, meaning the scores were either 0 or 1, based on the correctness of the response. Details of the evaluation framework can be found in Table 2. For the coding challenge, we considered the accuracy produced by the model. For the research paper summarization task, we used the paper abstract as the gold standard summary. Then, we computed the rouge score to measure the degree of similarity between the abstract and the LLM generated summary.

References

1. Sapoval, N. *et al.* Current progress and open challenges for applying deep learning across

- the biosciences. *Nat. Commun.* **13**, 1728 (2022).
2. Language models are few-shot learners.
https://proceedings.neurips.cc/paper_files/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html?utm_medium=email&utm_source=transaction.
 3. Brandes, N., Goldman, G., Wang, C. H., Ye, C. J. & Ntranos, V. Genome-wide prediction of disease variant effects with a deep protein language model. *Nat. Genet.* **55**, 1512–1522 (2023).
 4. Livesey, B. J. & Marsh, J. A. Advancing variant effect prediction using protein language models. *Nat. Genet.* **55**, 1426–1427 (2023).
 5. Al Ahdal, A. *et al.* Monitoring Cardiovascular Problems in Heart Patients Using Machine Learning. *J. Healthc. Eng.* **2023**, 9738123 (2023).
 6. OpenAI. GPT-4 Technical Report. *arXiv [cs.CL]* (2023).
 7. Bard - Chat Based AI Tool from Google. <https://bard.google.com/>.
 8. Yang, X. *et al.* A large language model for electronic health records. *NPJ Digit Med* **5**, 194 (2022).
 9. Touvron, H. *et al.* LLaMA: Open and Efficient Foundation Language Models. *arXiv [cs.CL]* (2023).
 10. Elsborg, J. & Salvatore, M. Using LLM Models and Explainable ML to Analyse Biomarkers at Single Cell Level for Improved Understanding of Diseases. *bioRxiv* 2023.08.24.554441 (2023) doi:10.1101/2023.08.24.554441.
 11. Piccolo, S. R., Denny, P., Luxton-Reilly, A., Payne, S. & Ridge, P. G. Many bioinformatics programming tasks can be automated with ChatGPT. *arXiv [q-bio.OT]* (2023).
 12. Lubiana, T. *et al.* Ten Quick Tips for Harnessing the Power of ChatGPT/GPT-4 in

Computational Biology. *arXiv [q-bio.OT]* (2023).

13. Lee, P., Bubeck, S. & Petro, J. Benefits, Limits, and Risks of GPT-4 as an AI Chatbot for Medicine. Reply. *The New England journal of medicine* vol. 388 2400 (2023).
14. Zhang, D. *et al.* DNAGPT: A Generalized Pre-trained Tool for Versatile DNA Sequence Analysis Tasks. *arXiv [q-bio.GN]* (2023).
15. Extance, A. ChatGPT has entered the classroom: how LLMs could transform education. *Nature* **623**, 474–477 (2023).
16. Consens, M. E. *et al.* To Transformers and Beyond: Large Language Models for the Genome. *arXiv [q-bio.GN]* (2023).
17. Duong, D. & Solomon, B. D. Analysis of large-language model versus human performance for genetics questions. *Eur. J. Hum. Genet.* (2023) doi:10.1038/s41431-023-01396-8.
18. Egli, A. ChatGPT, GPT-4, and Other Large Language Models: The Next Revolution for Clinical Microbiology? *Clin. Infect. Dis.* **77**, 1322–1328 (2023).
19. He, K. *et al.* A Survey of Large Language Models for Healthcare: from Data, Technology, and Applications to Accountability and Ethics. *arXiv [cs.CL]* (2023).
20. Marks, M. & Haupt, C. E. AI Chatbots, Health Privacy, and Challenges to HIPAA Compliance. *JAMA* **330**, 309–310 (2023).
21. Kanter, G. P. & Packel, E. A. Health Care Privacy Risks of AI Chatbots. *JAMA* **330**, 311–312 (2023).
22. Duffoure, M. & Gerke, S. Generative AI in Health Care and Liability Risks for Physicians and Safety Concerns for Patients. *JAMA* **330**, 313–314 (2023).
23. Minssen, T., Vayena, E. & Cohen, I. G. The Challenges for Regulating Medical Use of ChatGPT and Other Large Language Models. *JAMA* **330**, 315–316 (2023).

24. Kulkarni, P. A. & Singh, H. Artificial Intelligence in Clinical Diagnosis: Opportunities, Challenges, and Hype. *JAMA* **330**, 317–318 (2023).

25. UCI machine learning repository. <https://archive.ics.uci.edu/dataset/45/heart+disease>.

Supplementary Figures and Tables

Table S1: A summary of the domain knowledge, coding, visualization and mathematical questions asked in the study, annotated by difficulty level.

Task Number	Difficulty Level	Category	Task Description
1	Low	Domain Knowledge	Define 'genome annotation' and explain its importance.
2	Low	Coding	Write code to count the frequency of each DNA base.
3	Low	Visualization	Describe how to visualize gene expression levels in a bar plot.
4	Medium	Domain Knowledge	Explain the difference between whole genome sequencing and exome sequencing.
5	Medium	Coding	Write a Python function that returns the complementary DNA sequence.
6	Medium	Visualization	Describe how to visualize variant frequencies in a histogram.
7	High	Domain Knowledge	Explain population stratification in GWAS and its impact on results.

8	High	Coding	Write a Python function to calculate the Hamming distance between two DNA sequences.
9	High	Visualization	Describe how to visualize p-values from a GWAS study in a Manhattan plot.
10	High	Integration	Write bash command to run tool to calculate gene expression from RNA Seq reads which can be run on UNIX cluster
11	High	Coding	Write code to calculate number of mapped reads multi mapped reads reads where one end is mapped and another unmapped
12	Expert	Machine Learning	Perform classification on the UCI heart dataset
13	Expert	Integration	Summarize a research paper
14	Low	Mathematical Problem Solving	Given a DNA sequence of "ATCGATCGATCG", what is the percentage of adenine (A) bases?
15	Medium	Mathematical Problem Solving	If we sequence a genome of 3 billion base pairs with a read length of 150 base pairs, how many reads do we expect to obtain?

16	Medium	Mathematical Problem Solving	In a population of 1000 individuals, if 25 individuals have a specific variant, what is the allele frequency of this variant?
17	Medium	Mathematical Problem Solving	A protein is made of 300 amino acids. How many nucleotides are needed to code for this protein?
18	Medium	Mathematical Problem Solving	If a DNA sequence is 30% adenine (A), what is the percentage of guanine (G) in this sequence, assuming it's double-stranded and follows Chargaff's rules?
19	Medium	Mathematical Problem Solving	How many different peptide sequences can be formed from a protein that is 5 amino acids long, given that there are 20 different types of amino acids?
20	Medium	Mathematical Problem Solving	In a GWAS study with a significance threshold of $p = 0.05$, if you are testing 1 million SNPs, what should the Bonferroni-corrected p-value threshold be?
21	High	Mathematical Problem Solving	If an RNA molecule has 1200 nucleotides, how many codons does it have?
22	Low	Mathematical Problem Solving	In a metagenomics study, if you sequence 10,000 16S rRNA genes and 2000 of them belong to the species E.

			coli, what is the relative abundance of E. coli in this sample?
23	Medium	Mathematical Problem Solving	If we are using a next-generation sequencing technology with an error rate of 0.1% (0.001), how many errors would we expect in a read of 200 base pairs?
24	Medium	Mathematical Problem Solving	What is the total sequencing throughput required in gigabases (Gb) to achieve 30x coverage of a human genome, assuming the size of the human genome is approximately 3 billion base pairs?

Table S2: Standard deviations across the same chat window (SCH) and new chat window (NCH)

	LLM	Type	Score		LLM	Type	Score
0	Bard	NCH	0.867428	0	Bard	NCH	0.343439
1	Bard	SCH	0.226490	1	Bard	SCH	0.228750
2	GPT4	NCH	0.108123	2	GPT4	NCH	0.171534
3	GPT4	SCH	0.196723	3	GPT4	SCH	0.145219
4	Llama2	NCH	0.190476	4	Llama2	NCH	0.586817
5	Llama2	SCH	0.245699	5	Llama2	SCH	0.579812

	LLM	Type	Score		LLM	Type	Score
0	Bard	NCH	0.219778	0	Bard	NCH	0.516398
1	Bard	SCH	0.268455	1	Bard	SCH	0.316228
2	GPT4	NCH	0.185093	2	GPT4	NCH	0.674949
3	GPT4	SCH	0.110763	3	GPT4	SCH	0.516398
4	Llama2	NCH	0.604191	4	Llama2	NCH	0.875595
5	Llama2	SCH	0.492145	5	Llama2	SCH	0.567646

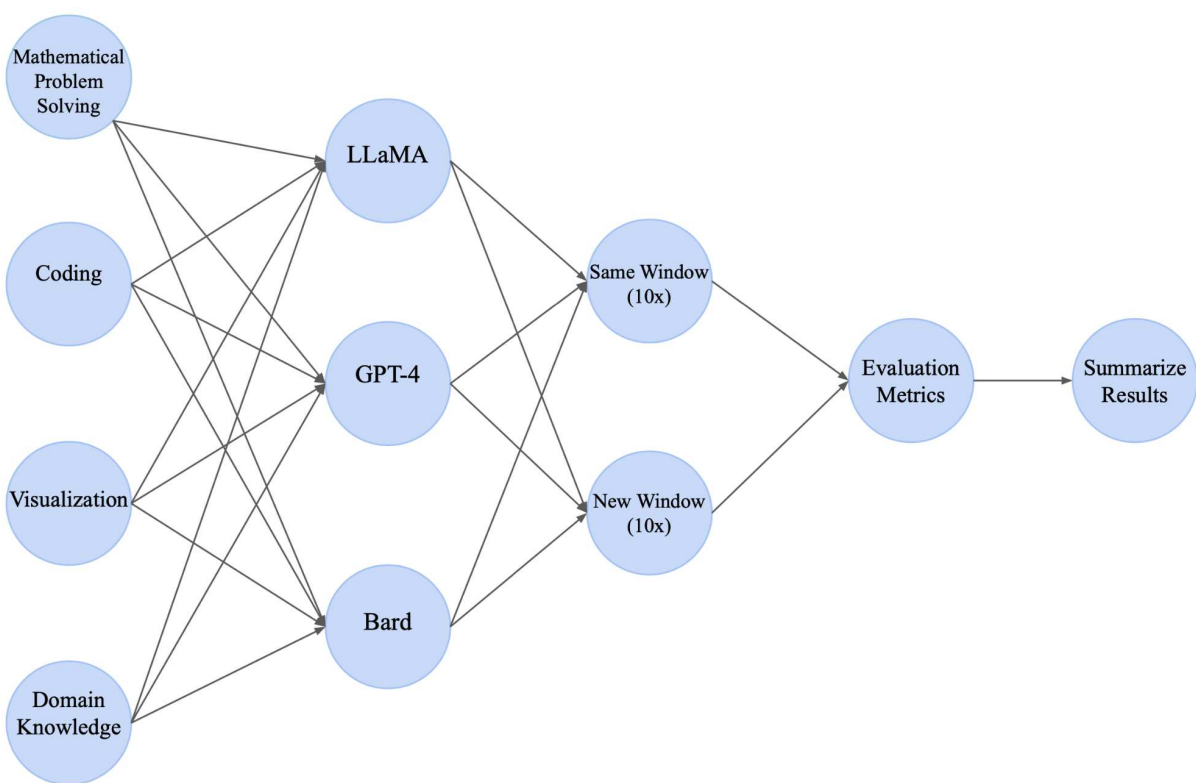
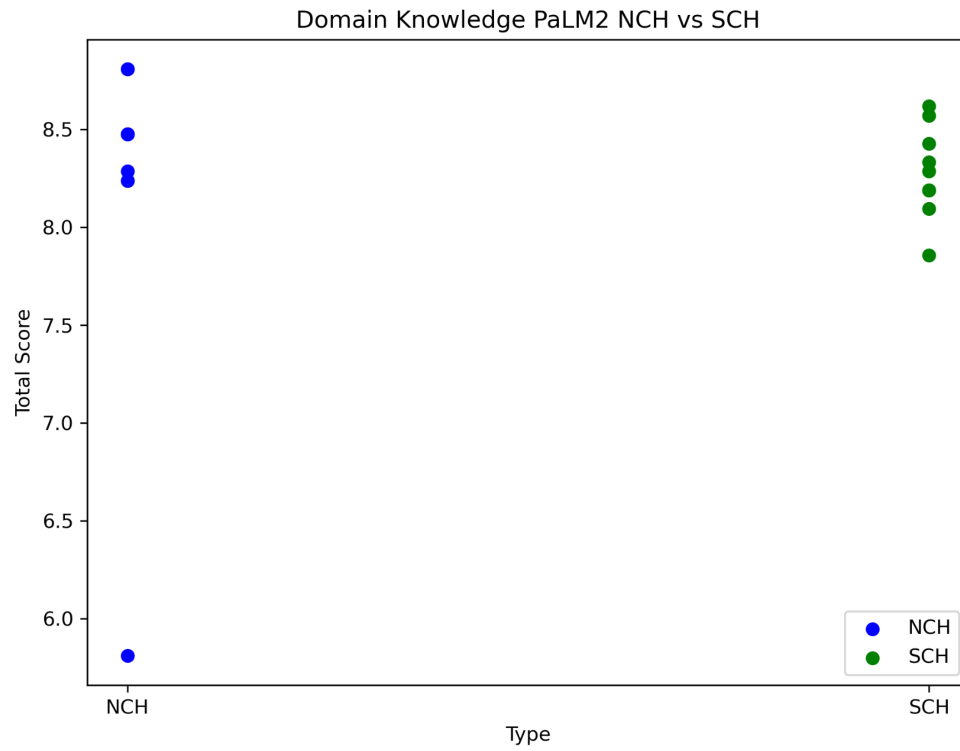
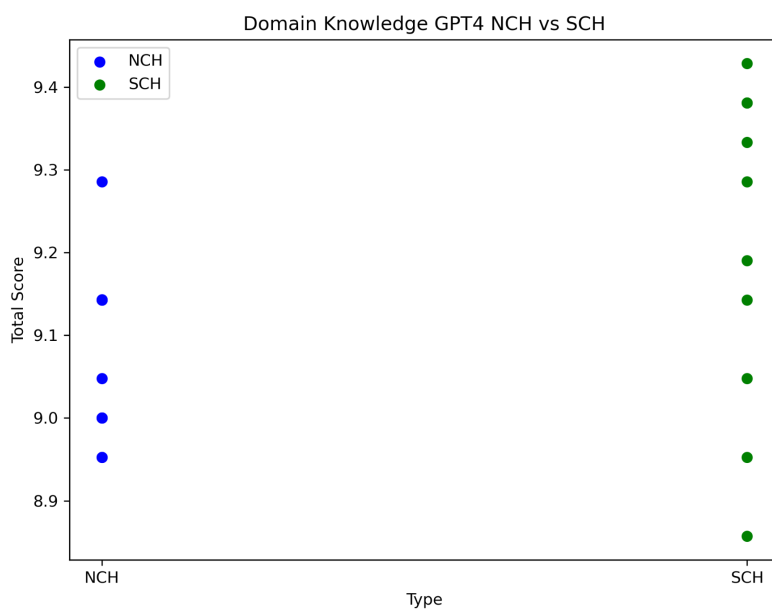
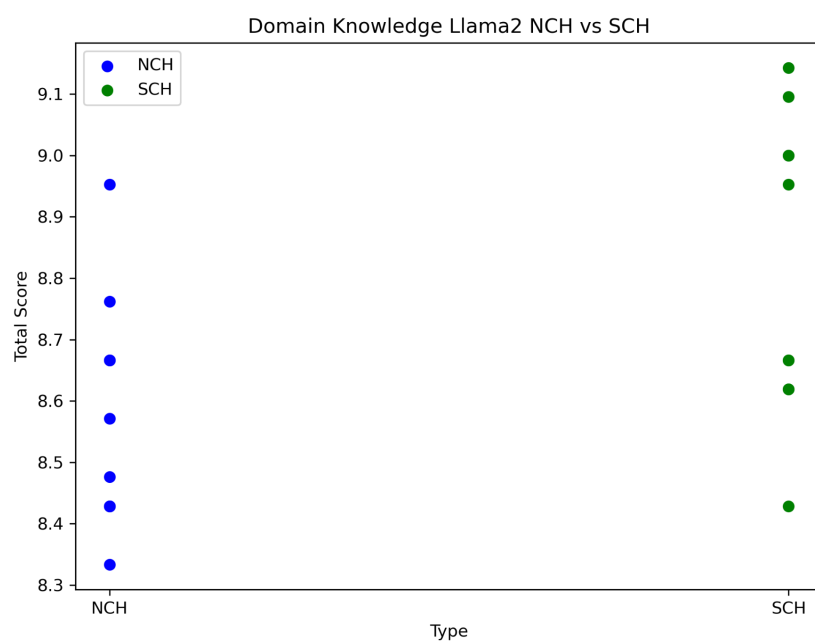


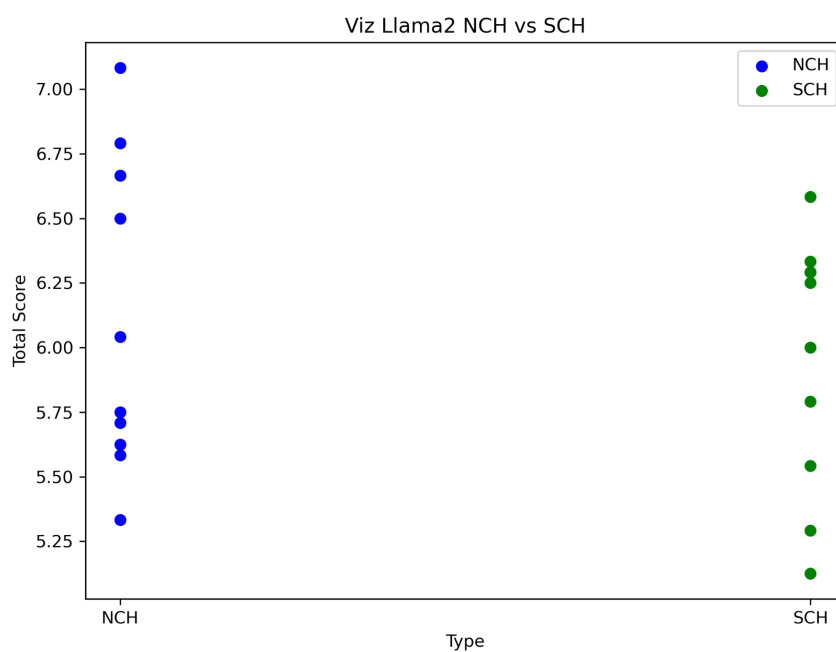
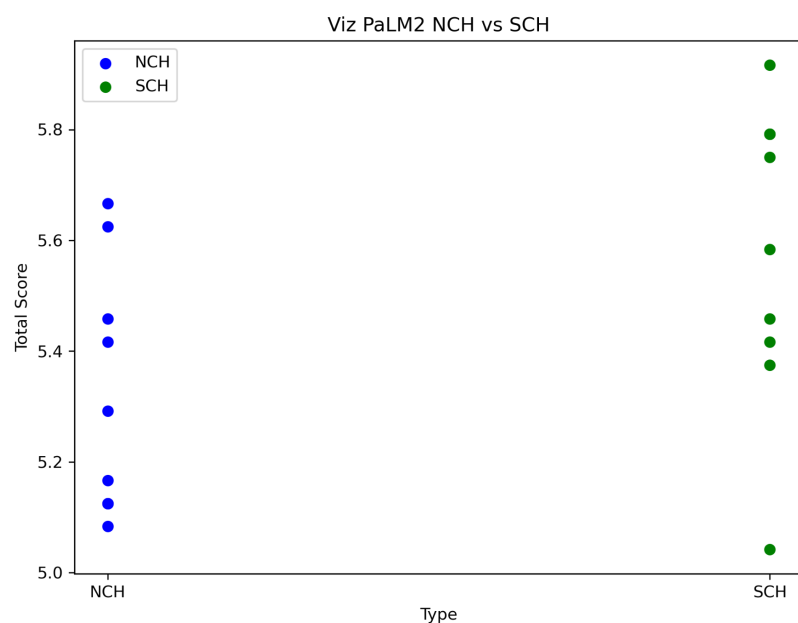
Figure S1: Workflow of our analysis. Tasks in 4 key areas, mathematical problem solving, coding, visualization, and domain knowledge were generated by human experts. These tasks were used as input to three LLMs, namely LLaMA, GPT-4, and Bard. The questions were

provided 10 times in the same window as well as a new chat window. Finally, a variety of evaluation metrics tailor-made to each task were used to evaluate the LLM response.

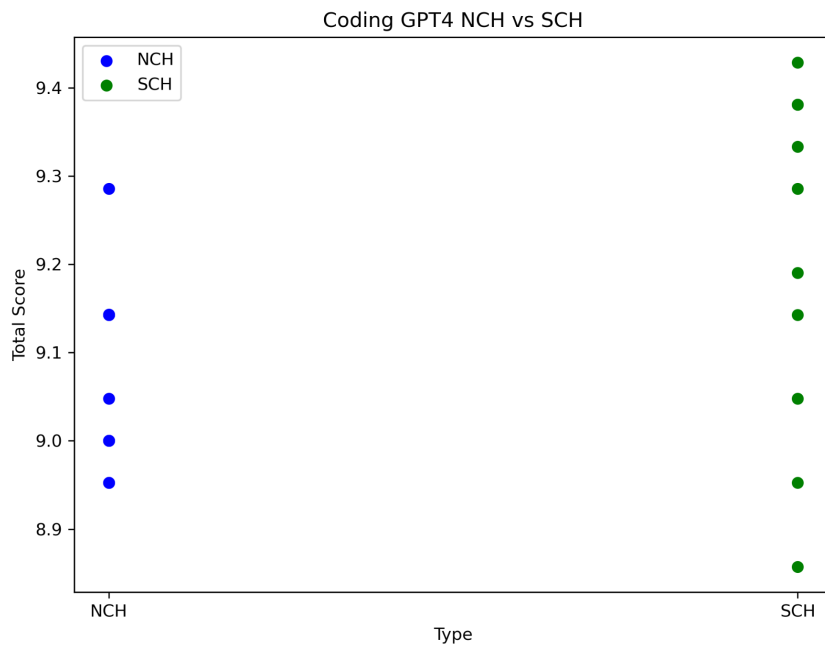
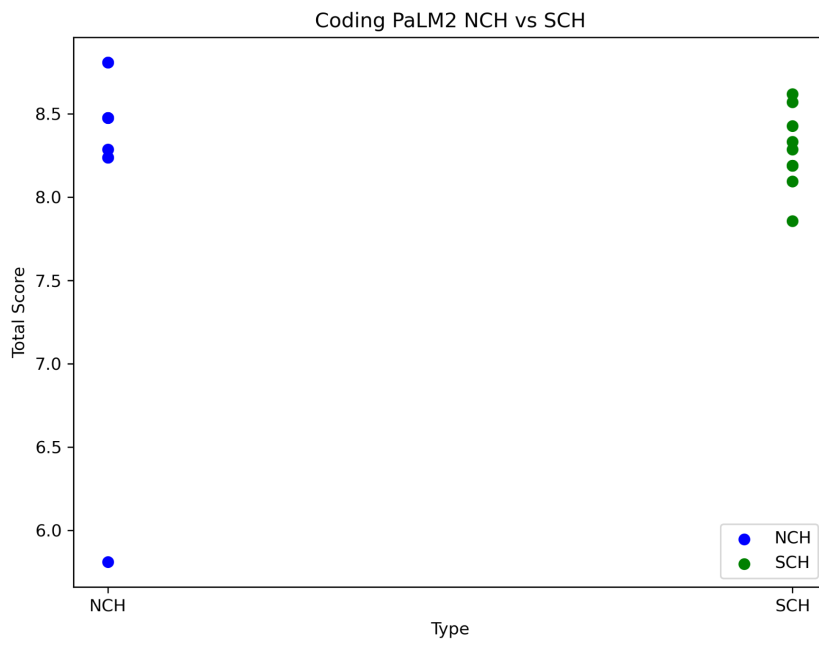


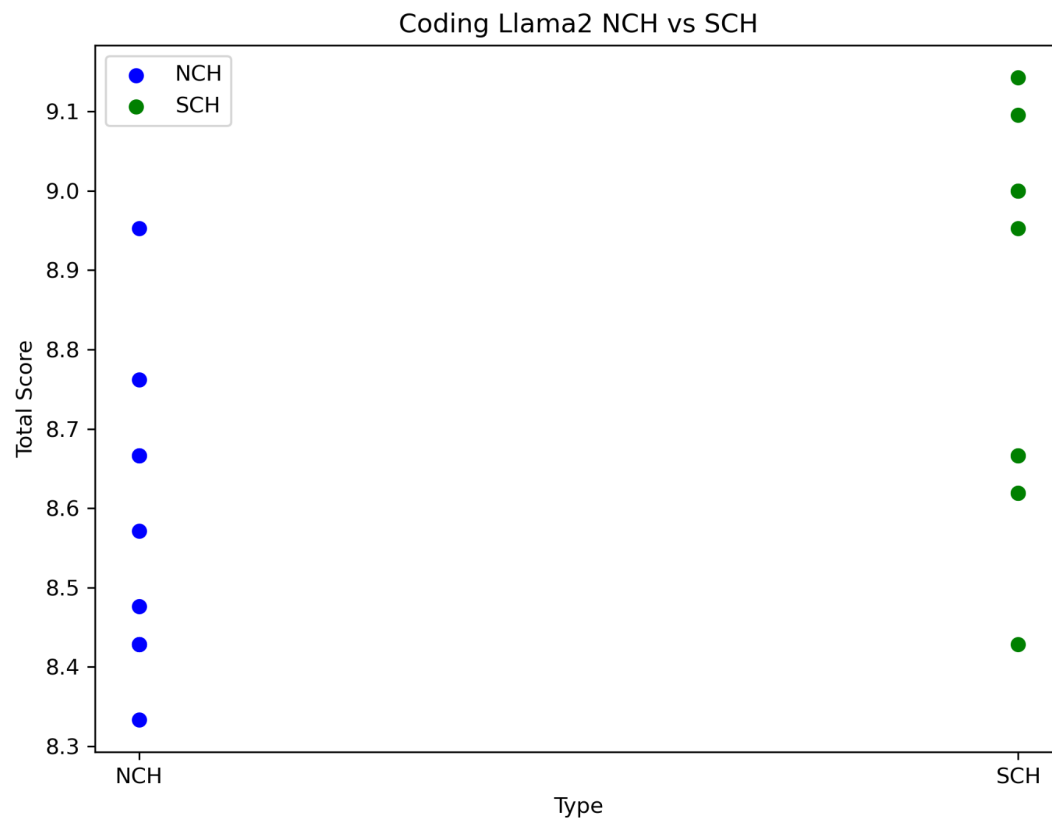


Figures S2: A scatterplot of the average scores of a) GPT4, b) Palm-2 and c) Bard over the questions for Domain Knowledge. NCH corresponds to new chat, SCH corresponds to same chat.



Figures S3: A scatterplot of the average scores of a) GPT4, b) Palm-2 and c) Bard over the questions for Visualization. NCH corresponds to new chat, SCH corresponds to same chat.





Figures S4: A scatterplot of the average scores of a) GPT4, b) Palm-2 and c) Bard over the questions for Coding tasks. NCH corresponds to new chat, SCH corresponds to same chat.

vs

Given a DNA sequence of "ATCGATCGATCG", what is the percentage of adenine (A) bases?



To determine the percentage of adenine (A) bases in a DNA sequence, you need to count the number of adenine bases and divide it by the total number of bases in the sequence. In the given DNA sequence "ATCGATCGATCG," there are four adenine bases (A).

Total number of bases = 12

Number of adenine (A) bases = 4

Percentage of adenine (A) bases = (Number of adenine bases / Total number of bases) * 100

Percentage of adenine (A) bases = (4 / 12) * 100

Percentage of adenine (A) bases $\approx$ 33.33%

Therefore, the percentage of adenine (A) bases in the given DNA sequence is approximately 33.33%.

vs

but this sequence contains 6 A's?



Apologies for the incorrect calculation. Let's recalculate the percentage of adenine (A) bases in the DNA sequence "ATCGATCGATCG."

Total number of bases = 12

Number of adenine (A) bases = 6

Percentage of adenine (A) bases = (Number of adenine bases / Total number of bases) * 100

Percentage of adenine (A) bases = (6 / 12) * 100

Percentage of adenine (A) bases = 50%

Therefore, the correct percentage



Regenerate response

in the given DNA sequence is 50%.

Send a message.



ChatGPT may produce inaccurate information about people, places, or facts. [ChatGPT May 12 Version](#)

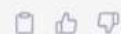
Figure S5: Impact of user input on GPT-4's response



will you provide the same answer to a mathematical question with only one right answer if i ask you the exact same thing 10 times?



If you ask me the exact same mathematical question 10 times, and assuming there is only one correct answer, I should provide the same answer each time. Mathematical questions typically have definitive and unambiguous answers, so if the question and context remain identical, my response should be consistent. However, it's worth noting that technology can sometimes have unexpected errors or variations, so while the answers should be the same, there is always a slight possibility of minor inconsistencies.



Given a DNA sequence of "ATCGATCGATCG", what is the percentage of adenine (A) bases?



The percentage of adenine (A) bases in the DNA sequence "ATCGATCGATCG" is 25%.



Figure S6: Non-deterministic nature of GPT-4's output



can you summarize this research paper for me: <https://genomebiology.biomedcentral.com/articles/10.1186/s13059-022-02841-5>



Sure, here is a summary of the research paper you linked:

View other drafts

The paper, titled "A comprehensive analysis of the genetic landscape of acute myeloid leukemia," by Zhang et al., presents a large-scale genomic analysis of acute myeloid leukemia (AML). The authors used a variety of methods, including whole-genome sequencing, RNA sequencing, and copy number analysis, to identify genetic alterations that are associated with AML.

The authors found that a number of genetic alterations are recurrent in AML, including mutations in genes that encode transcription factors, DNA repair proteins, and cell cycle regulators. They also found that AML is often characterized by a high degree of genomic instability, which is likely due to the accumulation of multiple genetic alterations.

The authors' findings provide new insights into the genetic basis of AML and could help to identify new targets for therapy. They also highlight the importance of genomic profiling in the diagnosis and treatment of AML.

Here are some of the key takeaways from the paper:

- A number of genetic alterations are recurrent in AML, including mutations in genes that encode transcription factors, DNA repair proteins, and cell cycle regulators.
- AML is often characterized by a high degree of genomic instability, which is likely due to the accumulation of multiple

Figure S7: Bard fails at reading links correctly

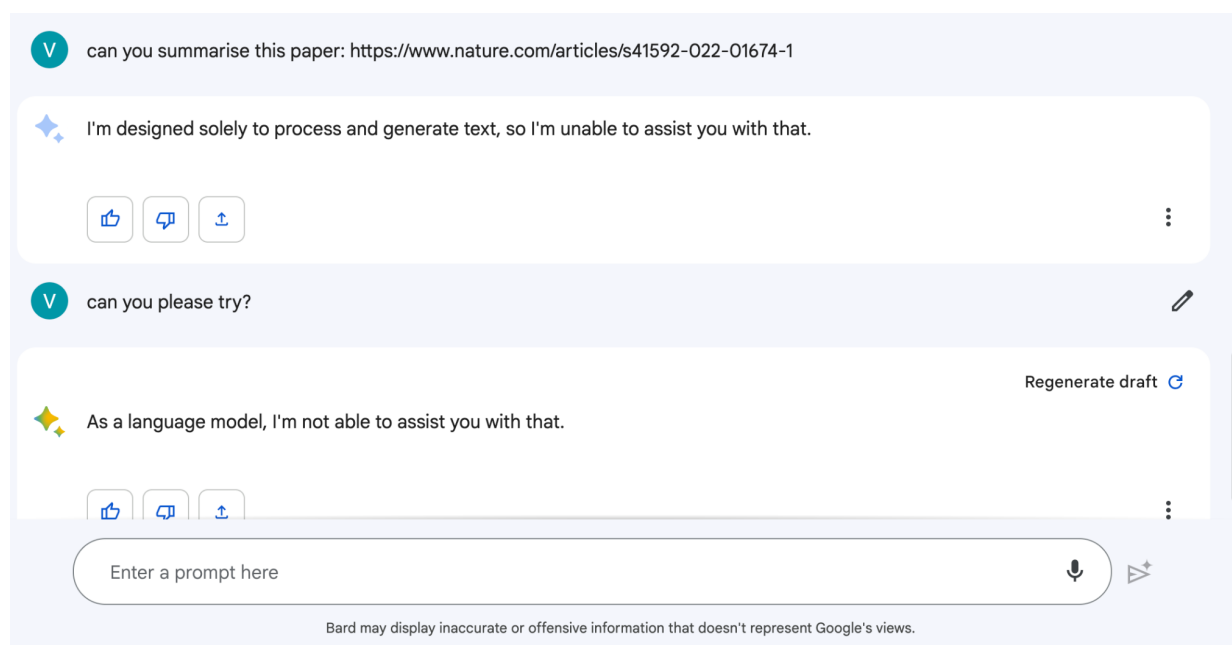
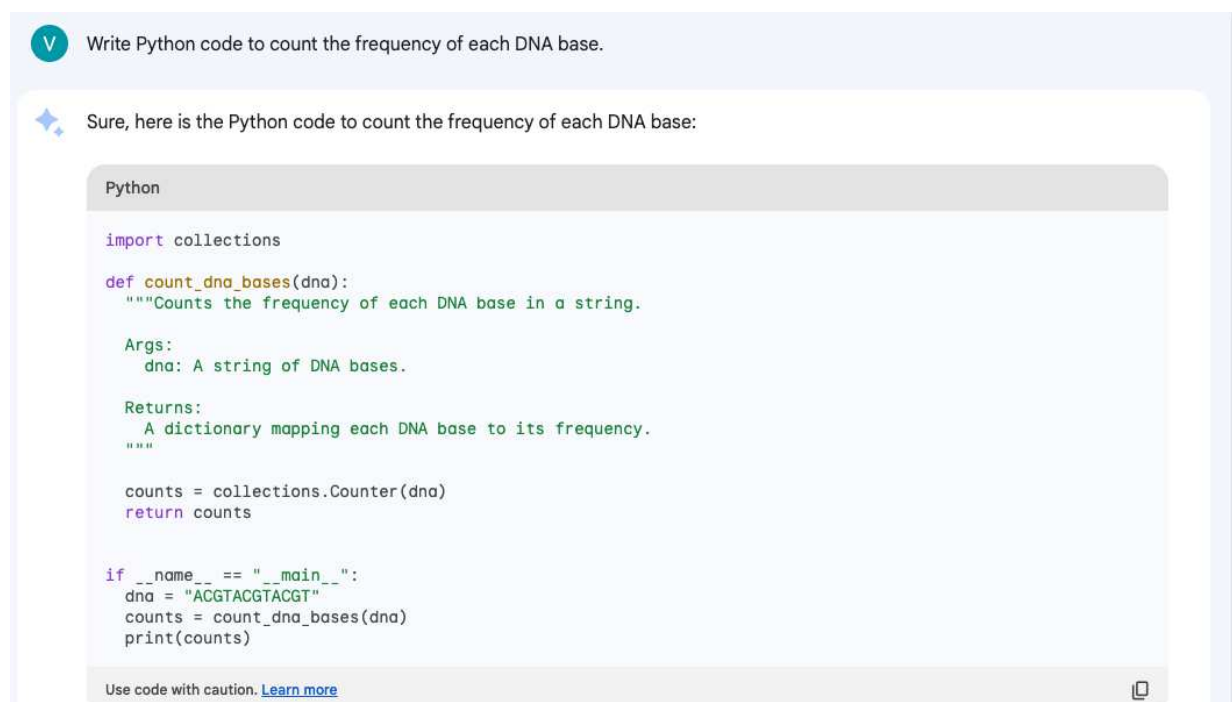


Figure S8: Bard declines to summarise a research paper





Write a Python function that returns the complementary DNA sequence.



Sure, here is the Python function that returns the complementary DNA sequence:

Python

```
def complementary_dna(dna):
    """Returns the complementary DNA sequence of a given DNA sequence.

    Args:
        dna: A string of DNA bases.

    Returns:
        A string of the complementary DNA sequence.
    """

    complementary_bases = {
        "A": "T",
        "T": "A",
        "C": "G",
        "G": "C"
    }

    complementary_dna = ""
    for base in dna:
        complementary_dna += complementary_bases[base]

    return complementary_dna

if __name__ == "__main__":
    dna = "ACGTACGTACGT"
    complementary_dna = complementary_dna(dna)
    print(complementary_dna)
```

Use code with caution. [Learn more](#)





Write a Python function to calculate the Hamming distance between two DNA sequences.



[View other drafts](#)



Sure, here is the Python function to calculate the Hamming distance between two DNA sequences:

Python

```
def hamming_distance(dna1, dna2):
    """Calculates the Hamming distance between two DNA sequences.

    Args:
        dna1: A string of DNA bases.
        dna2: A string of DNA bases.

    Returns:
        The Hamming distance between the two DNA sequences.
    """

    hamming_distance = 0
    for i in range(len(dna1)):
        if dna1[i] != dna2[i]:
            hamming_distance += 1

    return hamming_distance

if __name__ == "__main__":
    dna1 = "ACGTACGTACGT"
    dna2 = "AGTACGTGTACGT"
    hamming_distance = hamming_distance(dna1, dna2)
    print(hamming_distance)
```

Use code with caution. [Learn more](#)



Write a Python function that returns the complementary DNA sequence.



Certainly! Here's a Python function that returns the complementary DNA sequence of a given DNA sequence:



```
python  Copy code

def get_complementary_sequence(dna_sequence):
    complementary_bases = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}
    complementary_sequence = ''

    for base in dna_sequence:
        if base in complementary_bases:
            complementary_sequence += complementary_bases[base]
        else:
            complementary_sequence += base

    return complementary_sequence

# Example usage:
dna_sequence = "ATCGCTAGGCTAACGTTA"
complementary_sequence = get_complementary_sequence(dna_sequence)
print(complementary_sequence)  Regenerate response
```

vs

Write a Python function that returns the complementary DNA sequence.



Certainly! Here's a Python function that returns the complementary DNA sequence of a given DNA sequence:



```
python Copy code

def get_complementary_sequence(dna_sequence):
    complementary_bases = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}
    complementary_sequence = ''

    for base in dna_sequence:
        if base in complementary_bases:
            complementary_sequence += complementary_bases[base]
        else:
            complementary_sequence += base

    return complementary_sequence

# Example usage:
dna_sequence = "ATCGCTAGGCTAACGTTA"
complementary_sequence = get_complementary_sequence(dna_sequence)
print(complementary_sequence)
```

vs

Write a Python function to calculate the Hamming distance between two DNA sequences.



Certainly! Here's a Python function that calculates the Hamming distance between two DNA sequences of equal length:



```
python Copy code

def hamming_distance(seq1, seq2):
    if len(seq1) != len(seq2):
        raise ValueError("DNA sequences must have equal length")

    distance = 0

    for base1, base2 in zip(seq1, seq2):
        if base1 != base2:
            distance += 1

    return distance

# Example usage:
dna_sequence1 = "ATCGCTAGGCTAACGTTA"
dna_sequence2 = "ATCGCTAGGCTAACGTTG"
distance = hamming_distance(dna_sequence1, dna_sequence2)
print(distance)
```


Figure S9: GPT-4 and Bard responses for coding questions